

SDEV 2351

DAY 6: CENTRALIZED STATE, COMPONENT INTEGRATION, AND THE BYSRESULT
PATTERN

STORE VS. LOCAL — AND CLEANER ERROR HANDLING

Session Objectives

1. Apply a decision framework to determine when state should be centralized vs. kept local
2. Wire multiple Blazor components to subscribe to the same Fluxor store slice
3. Describe the BYSResult `Result<T>` API for explicit success/failure handling
4. Refactor a **service** from exception-throwing to `Result<T>`, accumulating multiple validation errors

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	BYSResult reading activation + store vs. local pair recall
Exploring	25 min	Decision framework, multi-component wiring, BYSResult API
Practicing	65 min	Account Overview wiring + service-to-Result refactor
Assessing	10 min	Exit ticket — store vs. local + Result API

PHASE 1 · 10 MINUTES

CONNECTING

Activating the BYSResult reading + store vs. local recall

BYSResult Reading Check

From the pre-class reading (BYSResult Pattern Guide):

- What does `result.IsSuccess` tell you?
- How do you add an error to a `Result<T>`?

Bridge: Day 5's effects used try/catch. The Result pattern gives us a cleaner contract — the service tells the effect whether it succeeded or failed, with structured errors.

Store vs. Local — Quick Recall

Think about Days 4-5. You built counter, todo, and product stores.

But not everything belongs in the store.

- **Store-worthy:** cart items, product catalog, user session
- **Local-only:** dropdown open/closed, form field focus, tooltip visibility

Name one piece of data for each category with your partner.

PHASE 2 · 25 MINUTES

EXPLORING

Decision framework, multi-component subscriptions, and BYSResult

Centralized vs. Local Decision Framework

CRITERION	CENTRALIZED (STORE)	LOCAL (COMPONENT)
Survives navigation?	Yes	No
Shared across components?	Yes	No
Needs time - travel debug?	Yes	No
Pure UI state?	No	Yes
Scope of change?	Cross-cutting	Single component

Rule of thumb: Centralization is a cost, not a goal. Promote to the store only when a second consumer appears.

Three Scenarios

1. **User's preferred language** — Centralized. Shared across all components, survives navigation, needs persistence.
2. **"Is this accordion section open?"** — Local. Pure UI state, single component, doesn't survive navigation.
3. **Shopping cart items** — Centralized. Shared between product pages and cart page, survives navigation, needs time-travel debug.

Multi-Component Subscription

Multiple components inject `IState<SameState>` — all re-render when the store updates.

NOTIFICATIONPANEL

```
@inject IState<DashboardState>
```

Displays notification count

PRODUCTSUMMARY

```
@inject IState<DashboardState>
```

Displays product count

One action dispatched, both components update. **No prop drilling, no event callbacks.**

BYSResult `Result<T>` API

Your toolkit for explicit success/failure handling:

METHOD / PROPERTY	PURPOSE
<code>new Result<T>()</code>	Create a new result
<code>.AddError(new Error("Category", "Message"))</code>	Add a validation error
<code>.WithValue(data)</code>	Set the success value
<code>.IsSuccess</code>	Check if no errors were added
<code>.IsFailure</code>	Check if errors exist
<code>.Value</code>	Get the success data
<code>.Errors</code>	Get the list of errors

Try/Catch vs. Result<T>

TRY/CATCH (DAY 5)

- Service throws exceptions
- Effect wraps in try/catch
- First error stops execution
- Good for infrastructure errors

RESULT<T> (DAY 6)

- Service returns Result<T>
- Effect checks IsSuccess/IsFailure
- Accumulates ALL errors before returning
- Good for business logic validation

They work together: `Result<T>` for business outcomes, try/catch around HTTP calls for infrastructure errors.

BYSResult + Fluxor Integration

```
[EffectMethod]
public async Task Handle(
    LoadProductsAction action, IDispatcher dispatcher)
{
    var result = await _svc.GetAllAsync();
    if (result.IsSuccess)
        dispatcher.Dispatch(new SuccessAction(result.Value));
    else
        dispatcher.Dispatch(new FailureAction(result.Errors));
}
```

The service returns `Result<T>` — no try/catch needed for business logic.

Full code: [see reference-sheet.md](#) or your IDE

Demonstration: Dashboard Wiring + Result Refactor

Both patterns built live in `Day06Demo`. The store, the three component shells and the service are already written — `TODO` markers sit only at the points typed on screen:

PART	WHAT HAPPENS
Demo A — wire 3 components	<code>@inherits FluxorComponent</code> + <code>@inject IState<DashboardState></code> on each
Demo A — loading/error block	<code>@if IsLoading / else if ErrorMessage / else</code> on all three, from shared state
Demo A — run	One <code>LoadDashboardAction</code> , all three cards re-render
Demo B — refactor service	<code>IDashboardService</code> / <code>MockDashboardService</code> → <code>Task<Result<DashboardData>></code>
Demo B — refactor effect	Replace try/catch with <code>result.IsSuccess</code> / <code>result.IsFailure</code>
Show — error accumulation	<code>AddAsync</code> collects all errors before returning

Practice 1 repeats the wiring on your own practice project; Practice 2 applies the `Result<T>` refactor to `ProductService`.

PHASE 3 · 65 MINUTES

PRACTICING

Account Overview multi-component wiring + service-to-Result refactor

Practice 1: Account Overview 3-Component Wiring (worked launch, 20 min)

Starter provides: `AccountState`, actions, reducers and effect all complete. Three card `.razor` files with UI but no Fluxor wiring. The parent `Account.razor` is **already built and already dispatches** `LoadAccountAction` — you do not add that.

You add to each of the three cards:

- `@inherits FluxorComponent`
- `@inject IState<AccountState>`
- Data bindings to display state values

Same multi-component wiring the demonstration just showed, applied to a different feature. One action, three cards re-render. No prop drilling.

Practice 2: Service-to-Result Refactor (Independent, 25 min)

Starter provides: A `ProductService` that throws exceptions on validation failure

You refactor (in order, each harder than the last):

1. `GetByIdAsync` — single validation + not-found (straightforward)
2. `AddAsync` — multi-error accumulation (name + price validation)

For each method:

- Change return type to `Result<T>`
- Replace `throw` with `result.AddError()`
- Return `result.WithValue()` on success
- Update the calling effect to check `result.IsSuccess`

A third method, `UpdateAsync`, is **take-home** if you run short on time.

Troubleshooting Tips

SYMPTOM	LIKELY CAUSE
Only one component updates	Other components missing <code>@inherits FluxorComponent</code>
"Object reference" on <code>State.Value</code>	Subscribing to wrong state type
<code>result.Value</code> throws exception	Accessing <code>.Value</code> without checking <code>IsSuccess</code> first
All errors not showing	Using <code>throw</code> (stops at first) instead of <code>AddError()</code> (accumulates)

PHASE 4 · 10 MINUTES

ASSESSING

Summary

Today you learned to:

1. **Apply a decision framework** — five criteria to decide store vs. local state
2. **Wire multiple components** to the same Fluxor store slice with `IState<T>`
3. **Describe the BYSResult API** — `Result<T>`, `IsSuccess`, `IsFailure`, `AddError()`, `WithValue()`
4. **Integrate Result<T> with Fluxor** — effects check `IsSuccess` instead of catching exceptions

Exit Ticket

Two questions (written):

1. A user's preferred language setting — Fluxor store or local component state? One sentence explaining why.
2. A service method returns `Result<Product>` but still has `throw new ArgumentException(...)` on line 5. Is this correct? Why or why not?

What's Next

Day 7: Route Guards and Navigation with State

- You now have a complete Fluxor toolkit: state, actions, reducers, effects, multi-component subscriptions
- Day 7 adds **route guards** — protecting pages based on state
- Does the user have items in their cart? Is the user authenticated?
- Route guards use the store to make navigation decisions

No pre-class reading assigned.